

Chapter 4. All About Functions

Dealing with Complexity

In Chapter 3, the programs we wrote only consisted of one section of code. However, if we wrote real programs like that, it would be impossible to maintain them. It would be really difficult to get multiple people working on the project, as any change in one part might adversely affect another part that another developer is working on.

To assist programmers in working together in groups, it is necessary to break programs apart into separate pieces, which communicate with each other through well-defined interfaces. This way, each piece can be developed and tested independently of the others, making it easier for multiple programmers to work on the project.

Programmers use *functions* to break their programs into pieces which can be independently developed and tested. Functions are units of code that do a defined piece of work on specified types of data. For example, in a word processor program, I may have a function called `handle_typed_character` which is activated whenever a user types in a key. The data the function uses would probably be the keypress itself and the document the user currently has open. The function would then modify the document according to the keypress it was told about.

The data items a function is given to process are called its *parameters*. In the word processing example, the key which was pressed and the document would be considered parameters to the `handle_typed_characters` function. The parameter list and the processing expectations of a function (what it is expected to do with the parameters) are called the function's interface. Much care goes into designing function interfaces, because if they are called from many places within a project, it is difficult to change them if necessary.

A typical program is composed of hundreds or thousands of functions, each with a

Chapter 4. All About Functions

small, well-defined task to perform. However, ultimately there are things that you cannot write functions for which must be provided by the system. Those are called *primitive functions* (or just *primitives*) - they are the basics which everything else is built off of. For example, imagine a program that draws a graphical user interface. There has to be a function to create the menus. That function probably calls other functions to write text, to write icons, to paint the background, calculate where the mouse pointer is, etc. However, ultimately, they will reach a set of primitives provided by the operating system to do basic line or point drawing. Programming can either be viewed as breaking a large program down into smaller pieces until you get to the primitive functions, or incrementally building functions on top of primitives until you get the large picture in focus. In assembly language, the primitives are usually the same thing as the system calls, even though system calls aren't true functions as we will talk about in this chapter.

How Functions Work

Functions are composed of several different pieces:

function name

A function's name is a symbol that represents the address where the function's code starts. In assembly language, the symbol is defined by typing the the function's name as a label before the function's code. This is just like labels you have used for jumping.

function parameters

A function's parameters are the data items that are explicitly given to the function for processing. For example, in mathematics, there is a sine function. If you were to ask a computer to find the sine of 2, sine would be the function's name, and 2 would be the parameter. Some functions have

many parameters, others have none.¹

local variables

Local variables are data storage that a function uses while processing that is thrown away when it returns. It's kind of like a scratch pad of paper.

Functions get a new piece of paper every time they are activated, and they have to throw it away when they are finished processing. Local variables of a function are not accessible to any other function within a program.

static variables

Static variables are data storage that a function uses while processing that is not thrown away afterwards, but is reused for every time the function's code is activated. This data is not accessible to any other part of the program.

Static variables are generally not used unless absolutely necessary, as they can cause problems later on.

global variables

Global variables are data storage that a function uses for processing which are managed outside the function. For example, a simple text editor may put the entire contents of the file it is working on in a global variable so it doesn't have to be passed to every function that operates on it.² Configuration values are also often stored in global variables.

return address

The return address is an "invisible" parameter in that it isn't directly used during the function. The return address is a parameter which tells the function

1. Function parameters can also be used to hold pointers to data that the function wants to send back to the program.

2. This is generally considered bad practice. Imagine if a program is written this way, and in the next version they decided to allow a single instance of the program edit multiple files. Each function would then have to be modified so that the file that was being manipulated would be passed as a parameter. If you had simply passed it as a parameter to begin with, most of your functions could have survived your upgrade unchanged.

Chapter 4. All About Functions

where to resume executing after the function is completed. This is needed because functions can be called to do processing from many different parts of your program, and the function needs to be able to get back to wherever it was called from. In most programming languages, this parameter is passed automatically when the function is called. In assembly language, the `call` instruction handles passing the return address for you, and `ret` handles using that address to return back to where you called the function from.

return value

The return value is the main method of transferring data back to the main program. Most programming languages only allow a single return value for a function.

These pieces are present in most programming languages. How you specify each piece is different in each one, however.

The way that the variables are stored and the parameters and return values are transferred by the computer varies from language to language as well. This variance is known as a language's *calling convention*, because it describes how functions expect to get and receive data when they are called.³

Assembly language can use any calling convention it wants to. You can even make one up yourself. However, if you want to interoperate with functions written in other languages, you have to obey their calling conventions. We will use the calling convention of the C programming language for our examples because it is the most widely used, and because it is the standard for Linux platforms.

3. A *convention* is a way of doing things that is standardized, but not forcibly so. For example, it is a convention for people to shake hands when they meet. If I refuse to shake hands with you, you may think I don't like you. Following conventions is important because it makes it easier for others to understand what you are doing, and makes it easier for programs written by multiple independent authors to work together.

Assembly-Language Functions using the C Calling Convention

You cannot write assembly-language functions without understanding how the computer's *stack* works. Each computer program that runs uses a region of memory called the stack to enable functions to work properly. Think of a stack as a pile of papers on your desk which can be added to indefinitely. You generally keep the things that you are working on toward the top, and you take things off as you are finished working with them.

Your computer has a stack, too. The computer's stack lives at the very top addresses of memory. You can push values onto the top of the stack through an instruction called `pushl`, which pushes either a register or memory value onto the top of the stack. Well, we say it's the top, but the "top" of the stack is actually the bottom of the stack's memory. Although this is confusing, the reason for it is that when we think of a stack of anything - dishes, papers, etc. - we think of adding and removing to the top of it. However, in memory the stack starts at the top of memory and grows downward due to architectural considerations. Therefore, when we refer to the "top of the stack" remember it's at the bottom of the stack's memory. You can also pop values off the top using an instruction called `popl`. This removes the top value from the stack and places it into a register or memory location of your choosing..

When we push a value onto the stack, the top of the stack moves to accomodate the additional value. We can actually continually push values onto the stack and it will keep growing further and further down in memory until we hit our code or data. So how do we know where the current "top" of the stack is? The stack register, `%esp`, always contains a pointer to the current top of the stack, wherever it is.

Every time we push something onto the stack with `pushl`, `%esp` gets subtracted by 4 so that it points to the new top of the stack (remember, each word is four bytes long, and the stack grows downward). If we want to remove something from the stack, we simply use the `popl` instruction, which adds 4 to `%esp` and puts the previous top value in whatever register you specified. `pushl` and `popl` each take

Chapter 4. All About Functions

one operand - the register to push onto the stack for `pushl`, or receive the data that is popped off the stack for `popl`.

If we simply want to access the value on the top of the stack without removing it, we can simply use the `%esp` register in indirect addressing mode. For example, the following code moves whatever is at the top of the stack into `%eax`:

```
movl (%esp), %eax
```

If we were to just do this:

```
movl %esp, %eax
```

then `%eax` would just hold the pointer to the top of the stack rather than the value at the top. Putting `%esp` in parenthesis causes the computer to go to indirect addressing mode, and therefore we get the value pointed to by `%esp`. If we want to access the value right below the top of the stack, we can simply issue this instruction:

```
movl 4(%esp), %eax
```

This instruction uses the base pointer addressing mode (see the Section called *Data Accessing Methods* in Chapter 2) which simply adds 4 to `%esp` before looking up the value being pointed to.

In the C language calling convention, the stack is the key element for implementing a function's local variables, parameters, and return address.

Before executing a function, a program pushes all of the parameters for the function onto the stack in the reverse order that they are documented. Then the program issues a `call` instruction indicating which function it wishes to start. The `call` instruction does two things. First it pushes the address of the next instruction, which is the return address, onto the stack. Then it modifies the instruction pointer (`%eip`) to point to the start of the function. So, at the time the function starts, the stack looks like this (the "top" of the stack is at the bottom on

this example):

```
Parameter #N
...
Parameter 2
Parameter 1
Return Address <--- (%esp)
```

Each of the parameters of the function have been pushed onto the stack, and finally the return address is there. Now the function itself has some work to do.

The first thing it does is save the current base pointer register, `%ebp`, by doing `pushl %ebp`. The base pointer is a special register used for accessing function parameters and local variables. Next, it copies the stack pointer to `%ebp` by doing `movl %esp, %ebp`. This allows you to be able to access the function parameters as fixed indexes from the base pointer. You may think that you can use the stack pointer for this. However, during your program you may do other things with the stack such as pushing arguments to other functions.

Copying the stack pointer into the base pointer at the beginning of a function allows you to always know where your parameters are (and as we will see, local variables too), even while you may be pushing things on and off the stack. `%ebp` will always be where the stack pointer was at the beginning of the function, so it is more or less a constant reference to the *stack frame* (the stack frame consists of all of the stack variables used within a function, including parameters, local variables, and the return address).

At this point, the stack looks like this:

```
Parameter #N    <--- N*4+4(%ebp)
...
Parameter 2     <--- 12(%ebp)
Parameter 1     <--- 8(%ebp)
Return Address  <--- 4(%ebp)
Old %ebp        <--- (%esp) and (%ebp)
```

Chapter 4. All About Functions

As you can see, each parameter can be accessed using base pointer addressing mode using the `%ebp` register.

Next, the function reserves space on the stack for any local variables it needs. This is done by simply moving the stack pointer out of the way. Let's say that we are going to need two words of memory to run a function. We can simply move the stack pointer down two words to reserve the space. This is done like this:

```
subl $8, %esp
```

This subtracts 8 from `%esp` (remember, a word is four bytes long).⁴ This way, we can use the stack for variable storage without worrying about clobbering them with pushes that we may make for function calls. Also, since it is allocated on the stack frame for this function call, the variable will only be alive during this function. When we return, the stack frame will go away, and so will these variables. That's why they are called local - they only exist while this function is being called.

Now we have two words for local storage. Our stack now looks like this:

Parameter #N	<--- N*4+4(%ebp)
...	
Parameter 2	<--- 12(%ebp)
Parameter 1	<--- 8(%ebp)
Return Address	<--- 4(%ebp)
Old %ebp	<--- (%ebp)
Local Variable 1	<--- -4(%ebp)
Local Variable 2	<--- -8(%ebp) and (%esp)

So we can now access all of the data we need for this function by using base pointer addressing using different offsets from `%ebp`. `%ebp` was made specifically for this purpose, which is why it is called the base pointer. You can use other registers in base pointer addressing mode, but the x86 architecture makes using the `%ebp` register a lot faster.

4. Just a reminder - the dollar sign in front of the eight indicates immediate mode addressing, meaning that we load the number 8 into `%esp` rather than the value at address 8.

Global variables and static variables are accessed just like we have been accessing memory in previous chapters. The only difference between the global and static variables is that static variables are only used by one function, while global variables are used by many functions. Assembly language treats them exactly the same, although most other languages distinguish them.

When a function is done executing, it does three things:

1. It stores its return value in `%eax`.
2. It resets the stack to what it was when it was called (it gets rid of the current stack frame and puts the stack frame of the calling code back into effect).
3. It returns control back to wherever it was called from. This is done using the `ret` instruction, which pops whatever value is at the top of the stack, and sets the instruction pointer, `%eip`, to that value.

So, before a function returns control to the code that called it, it must restore the previous stack frame. Note also that without doing this, `ret` wouldn't work, because in our current stack frame, the return address is not at the top of the stack. Therefore, before we return, we have to reset the stack pointer `%esp` and base pointer `%ebp` to what they were when the function began.

Therefore to return from the function you have to do the following:

```
movl %ebp, %esp
popl %ebp
ret
```

At this point, you should consider all local variables to be disposed of. The reason is that after you move the stack pointer back, future stack pushes will likely overwrite everything you put there. Therefore, you should never save the address of a local variable past the life of the function it was created in, or else it will be overwritten after the life of its stack frame ends.

Chapter 4. All About Functions

Control has now been handed back to the calling code, which can now examine `%eax` for the return value. The calling code also needs to pop off all of the parameters it pushed onto the stack in order to get the stack pointer back where it was (you can also simply add $4 * \text{number of parameters}$ to `%esp` using the `addl` instruction, if you don't need the values of the parameters anymore).⁵

Destruction of Registers

When you call a function, you should assume that everything currently in your registers will be wiped out. The only register that is guaranteed to be left with the value it started with is `%ebp`. `%eax` is guaranteed to be overwritten, and the others likely are. If there are registers you want to save before calling a function, you need to save them by pushing them on the stack before pushing the function's parameters. You can then pop them back off in reverse order after popping off the parameters. Even if you know a function does not overwrite a register you should save it, because future versions of that function may.

Other languages' calling conventions may be different. For example, other calling conventions may place the burden on the function to save any registers it uses. Be sure to check to make sure the calling conventions of your languages are compatible before trying to mix languages. Or in the case of assembly language, be sure you know how to call the other language's functions. them.

Extended Specification: Details of the C language calling convention (also known as the ABI, or Application Binary Interface) is available online. We have oversimplified and left out several important pieces to make this simpler for new programmers. For full details, you should check out the documents available at <http://www.linuxbase.org/spec/refspecs/> Specifically, you should

5. This is not always strictly needed unless you are saving registers on the stack before a function call. The base pointer keeps the stack frame in a reasonably consistent state. However, it is still a good idea, and is absolutely necessary if you are temporarily saving registers on the stack..

look for the *System V Application Binary Interface - Intel386 Architecture Processor Supplement*.

A Function Example

Let's take a look at how a function call works in a real program. The function we are going to write is the power function. We will give the power function two parameters - the number and the power we want to raise it to. For example, if we gave it the parameters 2 and 3, it would raise 2 to the power of 3, or $2*2*2$, giving 8. In order to make this program simple, we will only allow numbers 1 and greater.

The following is the code for the complete program. As usual, an explanation follows. Name the file `power.s`.

```
#PURPOSE:  Program to illustrate how functions work
#          This program will compute the value of
#           $2^3 + 5^2$ 
#

#Everything in the main program is stored in registers,
#so the data section doesn't have anything.
.section .data

.section .text

.globl _start
_start:
    pushl $3          #push second argument
    pushl $2          #push first argument
    call power        #call the function
    addl $8, %esp      #move the stack pointer back
```

Chapter 4. All About Functions

```
pushl %eax                #save the first answer before
                           #calling the next function

pushl $2                  #push second argument
pushl $5                  #push first argument
call power                #call the function
addl $8, %esp             #move the stack pointer back

popl %ebx                 #The second answer is already
                           #in %eax. We saved the
                           #first answer onto the stack,
                           #so now we can just pop it
                           #out into %ebx

addl %eax, %ebx           #add them together
                           #the result is in %ebx

movl $1, %eax             #exit (%ebx is returned)
int $0x80

#PURPOSE: This function is used to compute
#          the value of a number raised to
#          a power.
#
#INPUT:    First argument - the base number
#          Second argument - the power to
#                   raise it to
#
#OUTPUT:    Will give the result as a return value
#
#NOTES:     The power must be 1 or greater
#
#VARIABLES:
#          %ebx - holds the base number
#          %ecx - holds the power
```

```
#
#           -4(%ebp) - holds the current result
#
#           %eax is used for temporary storage
#
.type power, @function
power:
    pushl %ebp           #save old base pointer
    movl  %esp, %ebp     #make stack pointer the base pointer
    subl  $4, %esp       #get room for our local storage

    movl  8(%ebp), %ebx   #put first argument in %eax
    movl  12(%ebp), %ecx  #put second argument in %ecx

    movl  %ebx, -4(%ebp) #store current result

power_loop_start:
    cmpl  $1, %ecx       #if the power is 1, we are done
    je    end_power
    movl  -4(%ebp), %eax  #move the current result into %eax
    imull %ebx, %eax      #multiply the current result by
                        #the base number
    movl  %eax, -4(%ebp) #store the current result

    decl  %ecx           #decrease the power
    jmp   power_loop_start #run for the next power

end_power:
    movl  -4(%ebp), %eax  #return value goes in %eax
    movl  %ebp, %esp      #restore the stack pointer
    popl  %ebp           #restore the base pointer
    ret
```

Chapter 4. All About Functions

Type in the program, assemble it, and run it. Try calling `power` for different values, but remember that the result has to be less than 256 when it is passed back to the operating system. Also try subtracting the results of the two computations. Try adding a third call to the `power` function, and add its result back in.

The main program code is pretty simple. You push the arguments onto the stack, call the function, and then move the stack pointer back. The result is stored in `%eax`. Note that between the two calls to `power`, we save the first value onto the stack. This is because the only register that is guaranteed to be saved is `%ebp`. Therefore we push the value onto the stack, and pop the value back off after the second function call is complete.

Let's look at how the function itself is written. Notice that before the function, there is documentation as to what the function does, what its arguments are, and what it gives as a return value. This is useful for programmers who use this function. This is the function's interface. This lets the programmer know what values are needed on the stack, and what will be in `%eax` at the end.

We then have the following line:

```
.type power,@function
```

This tells the linker that the symbol `power` should be treated as a function. Since this program is only in one file, it would work just the same with this left out. However, it is good practice.

After that, we define the value of the `power` label:

```
power:
```

As mentioned previously, this defines the symbol `power` to be the address where the instructions following the label begin. This is how `call power` works. It transfers control to this spot of the program. The difference between `call` and `jmp` is that `call` also pushes the return address onto the stack so that the function can return, while the `jmp` does not.

Next, we have our instructions to set up our function:

```
pushl %ebp
movl  %esp, %ebp
subl  $4, %esp
```

At this point, our stack looks like this:

```
Base Number      <--- 12(%ebp)
Power            <--- 8(%ebp)
Return Address   <--- 4(%ebp)
Old %ebp         <--- (%ebp)
Current result   <--- -4(%ebp) and (%esp)
```

Although we could use a register for temporary storage, this program uses a local variable in order to show how to set it up. Often times there just aren't enough registers to store everything, so you have to offload them into local variables. Other times, your function will need to call another function and send it a pointer to some of your data. You can't have a pointer to a register, so you have to store it in a local variable in order to send a pointer to it.

Basically, what the program does is start with the base number, and store it both as the multiplier (stored in %ebx) and the current value (stored in -4(%ebp)). It also has the power stored in %ecx. It then continually multiplies the current value by the multiplier, decreases the power, and leaves the loop if the power (in %ecx) gets down to 1.

By now, you should be able to go through the program without help. The only things you should need to know is that `imull` does integer multiplication and stores the result in the second operand, and `decl` decreases the given register by 1. For more information on these and other instructions, see Appendix B.

A good project to try now is to extend the program so it will return the value of a number if the power is 0 (hint, anything raised to the zero power is 1). Keep trying. If it doesn't work at first, try going through your program by hand with a

scrap of paper, keeping track of where `%ebp` and `%esp` are pointing, what is on the stack, and what the values are in each register.

Recursive Functions

The next program will stretch your brains even more. The program will compute the *factorial* of a number. A factorial is the product of a number and all the numbers between it and one. For example, the factorial of 7 is $7*6*5*4*3*2*1$, and the factorial of 4 is $4*3*2*1$. Now, one thing you might notice is that the factorial of a number is the same as the product of a number and the factorial just below it. For example, the factorial of 4 is 4 times the factorial of 3. The factorial of 3 is 3 times the factorial of 2. 2 is 2 times the factorial of 1. The factorial of 1 is 1. This type of definition is called a recursive definition. That means, the definition of the factorial function includes the factorial function itself. However, since all functions need to end, a recursive definition must include a *base case*. The base case is the point where recursion will stop. Without a base case, the function would go on forever calling itself until it eventually ran out of stack space. In the case of the factorial, the base case is the number 1. When we hit the number 1, we don't run the factorial again, we just say that the factorial of 1 is 1. So, let's run through what we want the code to look like for our factorial function:

1. Examine the number
2. Is the number 1?
3. If so, the answer is one
4. Otherwise, the answer is the number times the factorial of the number minus one

This would be problematic if we didn't have local variables. In other programs, storing values in global variables worked fine. However, global variables only provide one copy of each variable. In this program, we will have multiple copies

of the function running at the same time, all of them needing their own copies of the data!⁶ Since local variables exist on the stack frame, and each function call gets its own stack frame, we are okay.

Let's look at the code to see how this works:

```
#PURPOSE - Given a number, this program computes the
#          factorial. For example, the factorial of
#          3 is 3 * 2 * 1, or 6. The factorial of
#          4 is 4 * 3 * 2 * 1, or 24, and so on.
#

#This program shows how to call a function recursively.

.section .data

#This program has no global data

.section .text

.globl _start
.globl factorial #this is unneeded unless we want to share
                 #this function among other programs
_start:
    pushl $4      #The factorial takes one argument - the
                 #number we want a factorial of. So, it
                 #gets pushed
    call factorial #run the factorial function
    addl $4, %esp #Scrubs the parameter that was pushed on
                 #the stack
    movl %eax, %ebx #factorial returns the answer in %eax, but
                   #we want it in %ebx to send it as our exit
                   #status
```

6. By "running at the same time" I am talking about the fact that one will not have finished before a new one is activated. I am not implying that their instructions are running at the same time.

Chapter 4. All About Functions

```
movl $1, %eax    #call the kernel's exit function
int $0x80

#This is the actual function definition
.type factorial,@function
factorial:
    pushl %ebp    #standard function stuff - we have to
                  #restore %ebp to its prior state before
                  #returning, so we have to push it
    movl %esp, %ebp #This is because we don't want to modify
                  #the stack pointer, so we use %ebp.

    movl 8(%ebp), %eax #This moves the first argument to %eax
                      #4(%ebp) holds the return address, and
                      #8(%ebp) holds the first parameter
    cmpl $1, %eax    #If the number is 1, that is our base
                  #case, and we simply return (1 is
                  #already in %eax as the return value)
    je end_factorial
    decl %eax        #otherwise, decrease the value
    pushl %eax       #push it for our call to factorial
    call factorial   #call factorial
    movl 8(%ebp), %ebx #%eax has the return value, so we
                  #reload our parameter into %ebx
    imull %ebx, %eax  #multiply that by the result of the
                  #last call to factorial (in %eax)
                  #the answer is stored in %eax, which
                  #is good since that's where return
                  #values go.

end_factorial:
    movl %ebp, %esp  #standard function return stuff - we
                  #have to restore %ebp and %esp to where
                  #they were before the function started
    ret             #return to the function (this pops the
```

```
#return value, too)
```

Assemble, link, and run it with these commands:

```
as factorial.s -o factorial.o
ld factorial.o -o factorial
./factorial
echo $?
```

This should give you the value 24. 24 is the factorial of 4, you can test it out yourself with a calculator: $4 * 3 * 2 * 1 = 24$.

I'm guessing you didn't understand the whole code listing. Let's go through it a line at a time to see what is happening.

```
_start:
    pushl $4
    call factorial
```

Okay, this program is intended to compute the factorial of the number 4. When programming functions, you are supposed to put the parameters of the function on the top of the stack right before you call it. Remember, a function's *parameters* are the data that you want the function to work with. In this case, the factorial function takes 1 parameter - the number you want the factorial of.

The `pushl` instruction puts the given value at the top of the stack. The `call` instruction then makes the function call.

Next we have these lines:

```
    addl $4, %esp
    movl %eax, %ebx
    movl $1, %eax
```

Chapter 4. All About Functions

```
int    $0x80
```

This takes place after `factorial` has finished and computed the factorial of 4 for us. Now we have to clean up the stack. The `addl` instruction moves the stack pointer back to where it was before we pushed the \$4 onto the stack. You should always clean up your stack parameters after a function call returns.

The next instruction moves `%eax` to `%ebx`. What's in `%eax`? It is `factorial`'s return value. In our case, it is the value of the factorial function. With 4 as our parameter, 24 should be our return value. Remember, return values are always stored in `%eax`. We want to return this value as the status code to the operating system. However, Linux requires that the program's exit status be stored in `%ebx`, not `%eax`, so we have to move it. Then we do the standard exit system call.

The nice thing about function calls is that:

- Other programmers don't have to know anything about them except it's arguments to use them.
- They provide standardized building blocks from which you can form a program.
- They can be called multiple times and from multiple locations and they always know how to get back to where they were since `call` pushes the return address onto the stack.

These are the main advantages of functions. Larger programs also use functions to break down complex pieces of code into smaller, simpler ones. In fact, almost all of programming is writing and calling functions.

Let's now take a look at how the `factorial` function itself is implemented.

Before the function starts, we have this directive:

```
.type factorial,@function  
factorial:
```

The `.type` directive tells the linker that `factorial` is a function. This isn't really needed unless we were using `factorial` in other programs. We have included it for completeness. The line that says `factorial:` gives the symbol `factorial` the storage location of the next instruction. That's how `call` knew where to go when we said `call factorial`.

The first real instructions of the function are:

```
pushl %ebp
movl  %esp, %ebp
```

As shown in the previous program, this creates the stack frame for this function. These two lines will be the way you should start every function.

The next instruction is this:

```
movl  8(%ebp), %eax
```

This uses base pointer addressing to move the first parameter of the function into `%eax`. Remember, `(%ebp)` has the old `%ebp`, `4(%ebp)` has the return address, and `8(%ebp)` is the location of the first parameter to the function. If you think back, this will be the value 4 on the first call, since that was what we pushed on the stack before calling the function (with `pushl $4`). parameter into `%eax`. As this function calls itself, it will have other values, too.

Next, we check to see if we've hit our base case (a parameter of 1). If so, we jump to the instruction at the label `end_factorial`, where it will be returned. It's already in `%eax` which we mentioned earlier is where you put return values. That is accomplished by these lines:

```
cmpl $1, %eax
je  end_factorial
```

If it's not our base case, what did we say we would do? We would call the `factorial` function again with our parameter minus one. So, first we decrease `%eax` by one:

Chapter 4. All About Functions

```
decl %eax
```

`decl` stands for decrement. It subtracts 1 from the given register or memory location (`%eax` in our case). `incl` is the inverse - it adds 1. After decrementing `%eax` we push it onto the stack since it's going to be the parameter of the next function call. And then we call `factorial` again!

```
pushl %eax  
call factorial
```

Okay, now we've called `factorial`. One thing to remember is that after a function call, we can never know what the registers are (except `%esp` and `%ebp`). So even though we had the value we were called with in `%eax`, it's not there any more. Therefore, we need pull it off the stack from the same place we got it the first time (at `8(%ebp)`). So, we do this:

```
movl 8(%ebp), %ebx
```

Now, we want to multiply that number with the result of the `factorial` function. If you remember our previous discussion, the result of functions are left in `%eax`. So, we need to multiply `%ebx` with `%eax`. This is done with this instruction:

```
imull %ebx, %eax
```

This also stores the result in `%eax`, which is exactly where we want the return value for the function to be! Since the return value is in place we just need to leave the function. If you remember, at the start of the function we pushed `%ebp`, and moved `%esp` into `%ebp` to create the current stack frame. Now we reverse the operation to destroy the current stack frame and reactivate the last one:

```
end_factorial:  
    movl %ebp, %esp  
    popl %ebp
```

Now we're already to return, so we issue the following command

`ret`

This pops the top value off of the stack, and then jumps to it. If you remember our discussion about `call`, we said that `call` first pushed the address of the next instruction onto the stack before it jumped to the beginning of the function. So, here we pop it back off so we can return there. The function is done, and we have our answer!

Like our previous program, you should look over the program again, and make sure you know what everything does. Look back through this section and the previous sections for the explanation of anything you don't understand. Then, take a piece of paper, and go through the program step-by-step, keeping track of what the values of the registers are at each step, and what values are on the stack. Doing this should deepen your understanding of what is going on.

Review

Know the Concepts

- What are primitives?
- What are calling conventions?
- What is the stack?
- How do `pushl` and `popl` affect the stack? What special-purpose register do they affect?
- What are local variables and what are they used for?
- Why are local variables so necessary in recursive functions?
- What are `%ebp` and `%esp` used for?
- What is a stack frame?

Use the Concepts

- Write a function called `square` which receives one argument and returns the square of that argument.
- Write a program to test your `square` function.
- Convert the maximum program given in the Section called *Finding a Maximum Value* in Chapter 3 so that it is a function which takes a pointer to several values and returns their maximum. Write a program that calls `maximum` with 3 different lists, and returns the result of the last one as the program's exit status code.
- Explain the problems that would arise without a standard calling convention.

Going Further

- Do you think it's better for a system to have a large set of primitives or a small one, assuming that the larger set can be written in terms of the smaller one?
- The factorial function can be written non-recursively. Do so.
- Find an application on the computer you use regularly. Try to locate a specific feature, and practice breaking that feature out into functions. Define the function interfaces between that feature and the rest of the program.
- Come up with your own calling convention. Rewrite the programs in this chapter using it. An example of a different calling convention would be to pass parameters in registers rather than the stack, to pass them in a different order, to return values in other registers or memory locations. Whatever you pick, be consistent and apply it throughout the whole program.
- Can you build a calling convention without using the stack? What limitations might it have?

- What test cases should we use in our example program to check to see if it is working properly?

Chapter 4. All About Functions